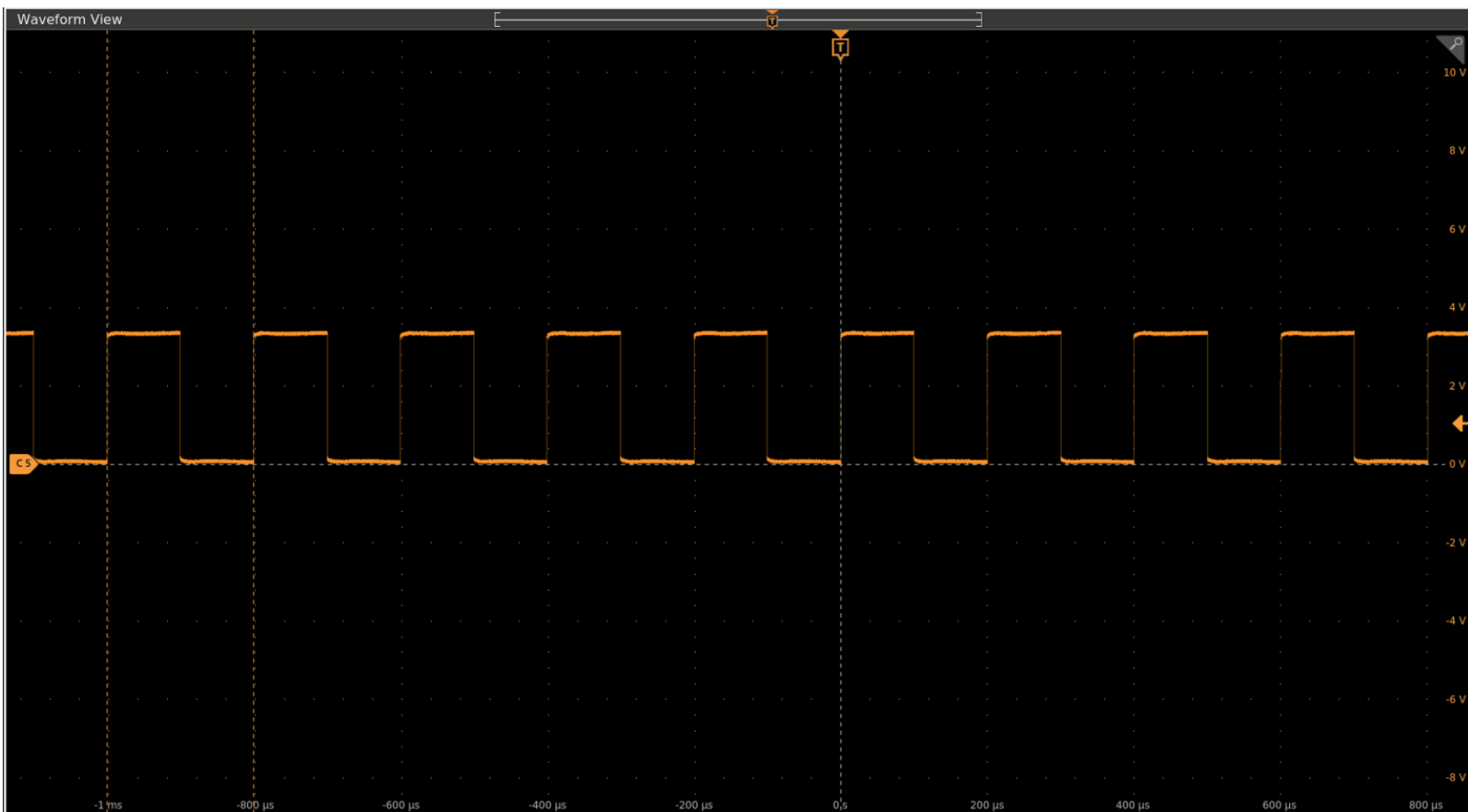
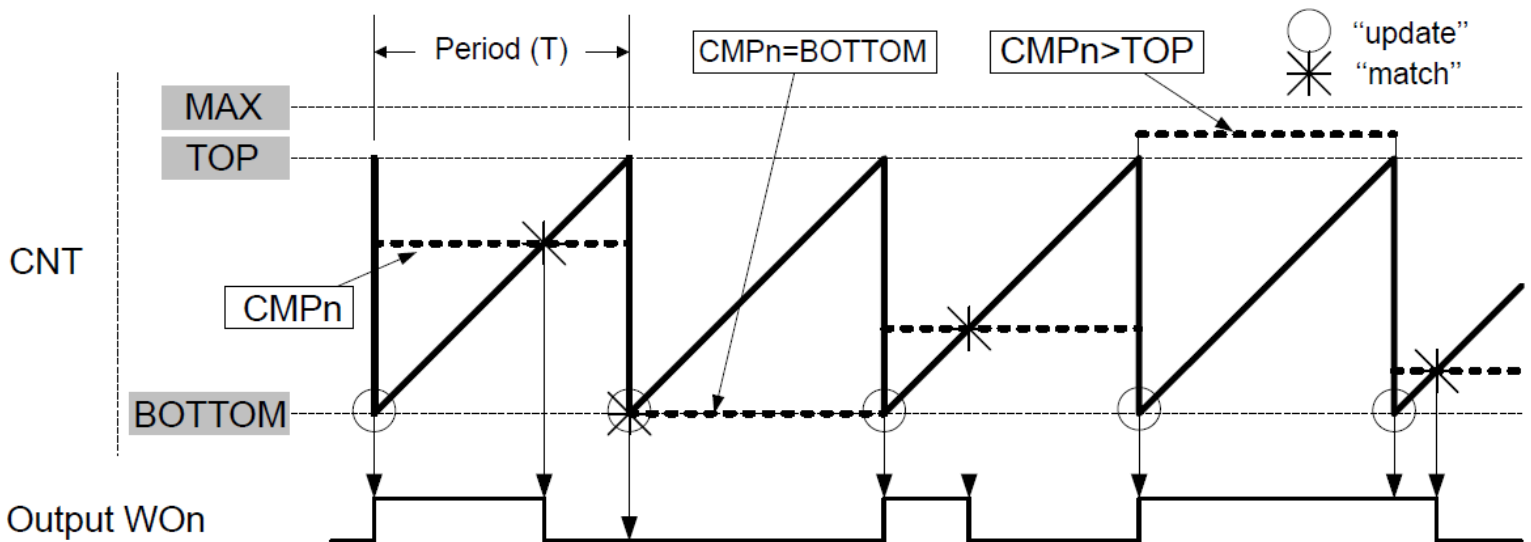


“Bare-Metal Embedded Systems with AVR128DA48 Course”

Day 7 - TCA Single Mode (PWM Basics)



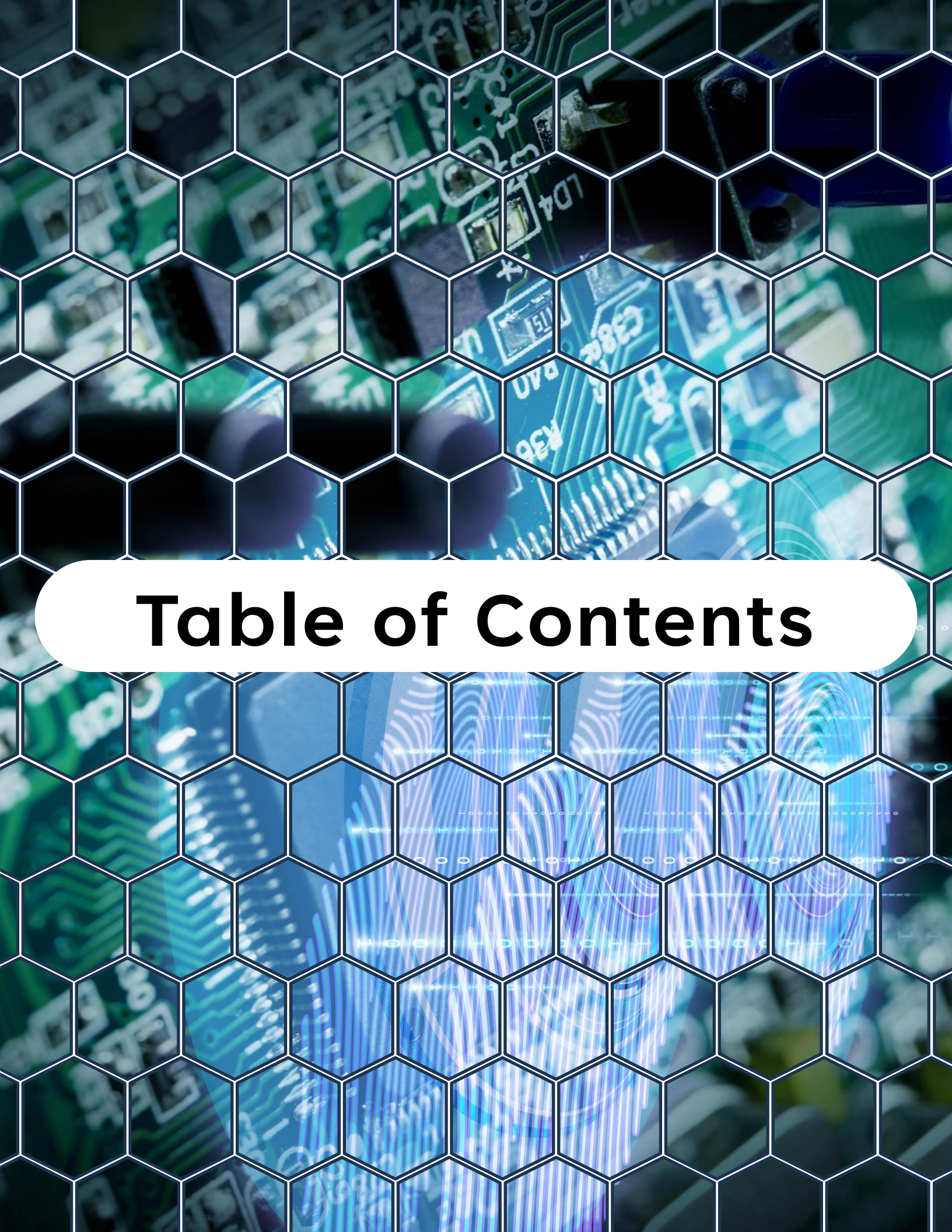
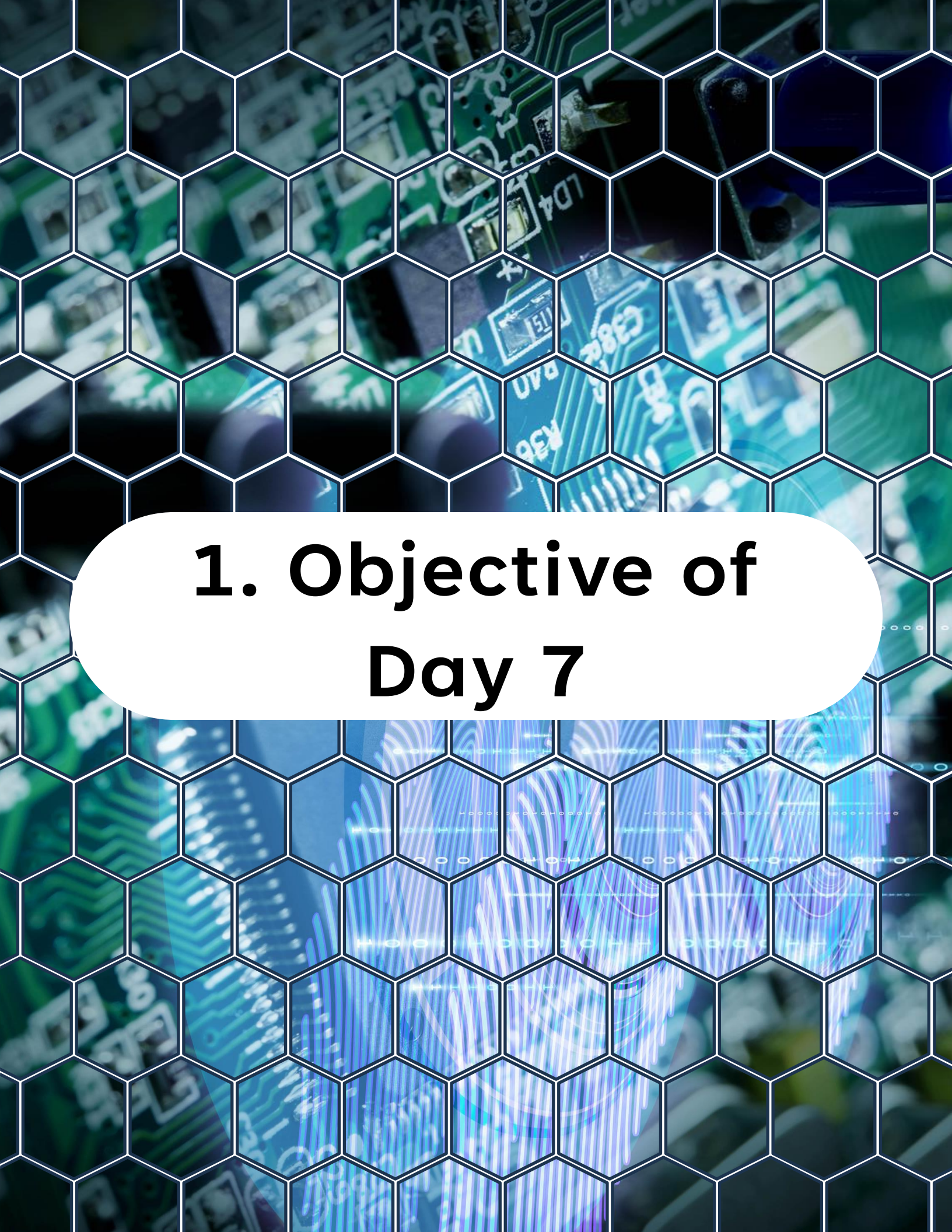


Table of Contents

Table of Contents

1. Objective of Day 7
2. PWM Fundamentals (What Actually Happens)
3. Timer Counter A (TCA) in Single Mode
4. TCA Registers You Will Use
5. PWM Frequency Math
6. Duty Cycle Math
7. Output Routing (PORTMUX)
8. Practical Lab - Generate PWM on WO0
9. Code Example - TCA0 Single Mode PWM (WO0)
10. Updating Duty Cycle at Runtime
11. Full Example
12. Debugging PWM Problems
13. What You Learned Today
14. What Comes Next



1. Objective of Day 7

1. Objective of Day 7

Today you will generate real hardware PWM using TCA in Single mode.

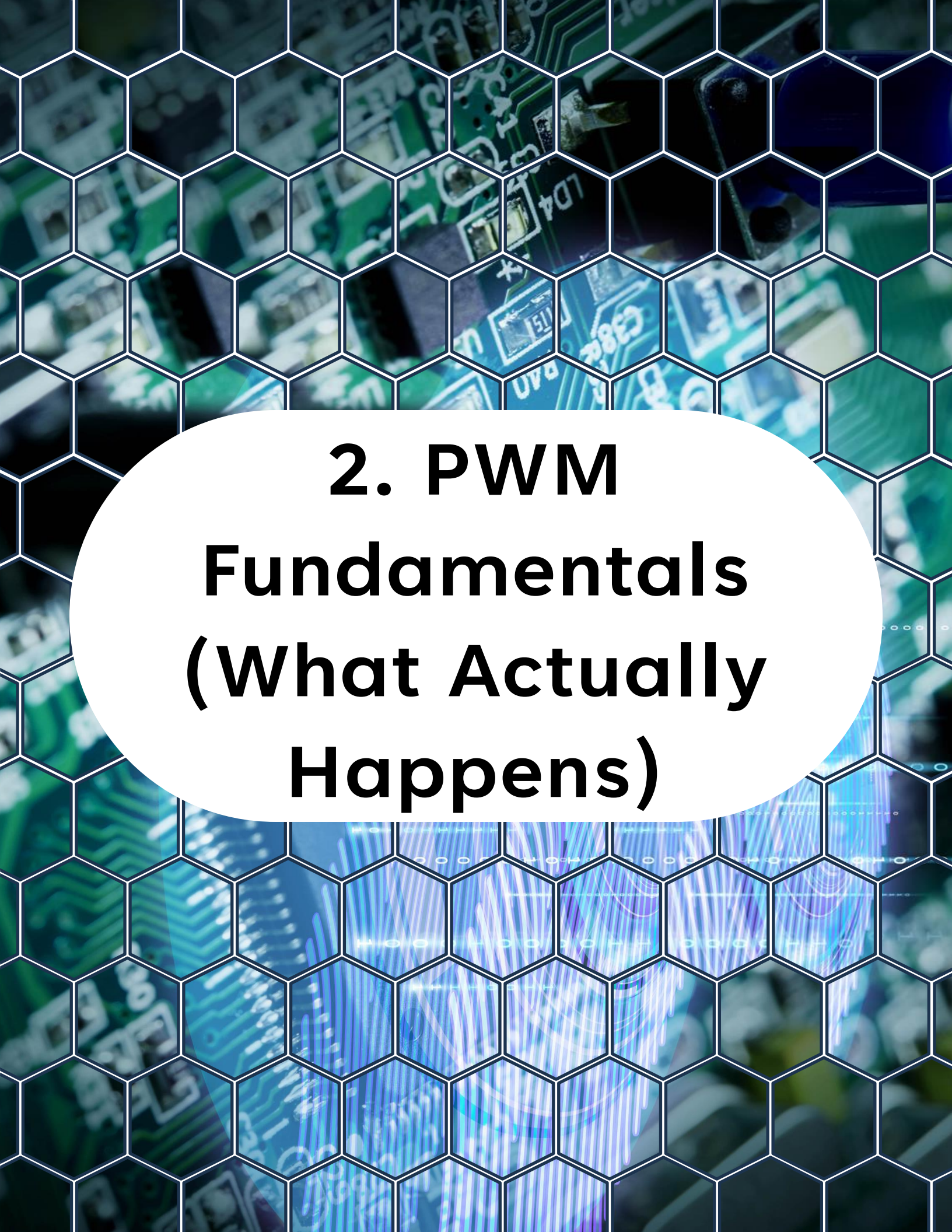
PWM is one of the most common embedded techniques because it lets you control analog-ish behavior using digital hardware:

- LED brightness control
- Motor speed control
- Power regulation (buck/boost, gate drive modulation)
- Audio tone generation (basic)
- Fan control

1. Objective of Day 7

By the end of this day, you will:

- Understand how TCA Single mode works
- Configure PWM frequency and duty cycle at register level
- Route TCA outputs to pins using PORTMUX
- Generate a stable PWM waveform on a real pin
- Update duty cycle safely at runtime



2. PWM Fundamentals (What Actually Happens)

2. PWM Fundamentals (What Actually Happens)

PWM is a repeating digital waveform with:

- Period (how often it repeats)
- Duty cycle (percentage of time high)

Key definitions:

- **Period** = total PWM cycle time
- **Duty cycle** = ON time / Period

If you keep frequency fixed and change duty cycle, you control average power delivered to a load.

For LEDs, the brightness changes (thanks to persistence of vision).

For motors, speed changes (average voltage/current delivered).

The background of the slide features a close-up, slightly blurred image of a green printed circuit board (PCB). The board is populated with various electronic components, including integrated circuits and resistors. A white hexagonal grid pattern is overlaid on the entire image, creating a honeycomb-like texture. In the center, a white circular area contains the main title text.

3. Timer Counter A (TCA) in Single Mode

3. Timer Counter A (TCA) in Single Mode

On the **AVR128DA48**, TCA is a flexible timer that can generate multiple PWM outputs using compare channels.

Single mode characteristics:

- One 16-bit counter
- One PER (period) register
- Multiple compare registers (CMP0, CMP1, CMP2)
- Multiple waveform outputs (WO0, WO1, WO2)

This means you can generate up to three PWM channels from one timer instance, all sharing the same frequency but different duty cycles.



4. TCA Registers You Will Use

4. TCA Registers You Will Use

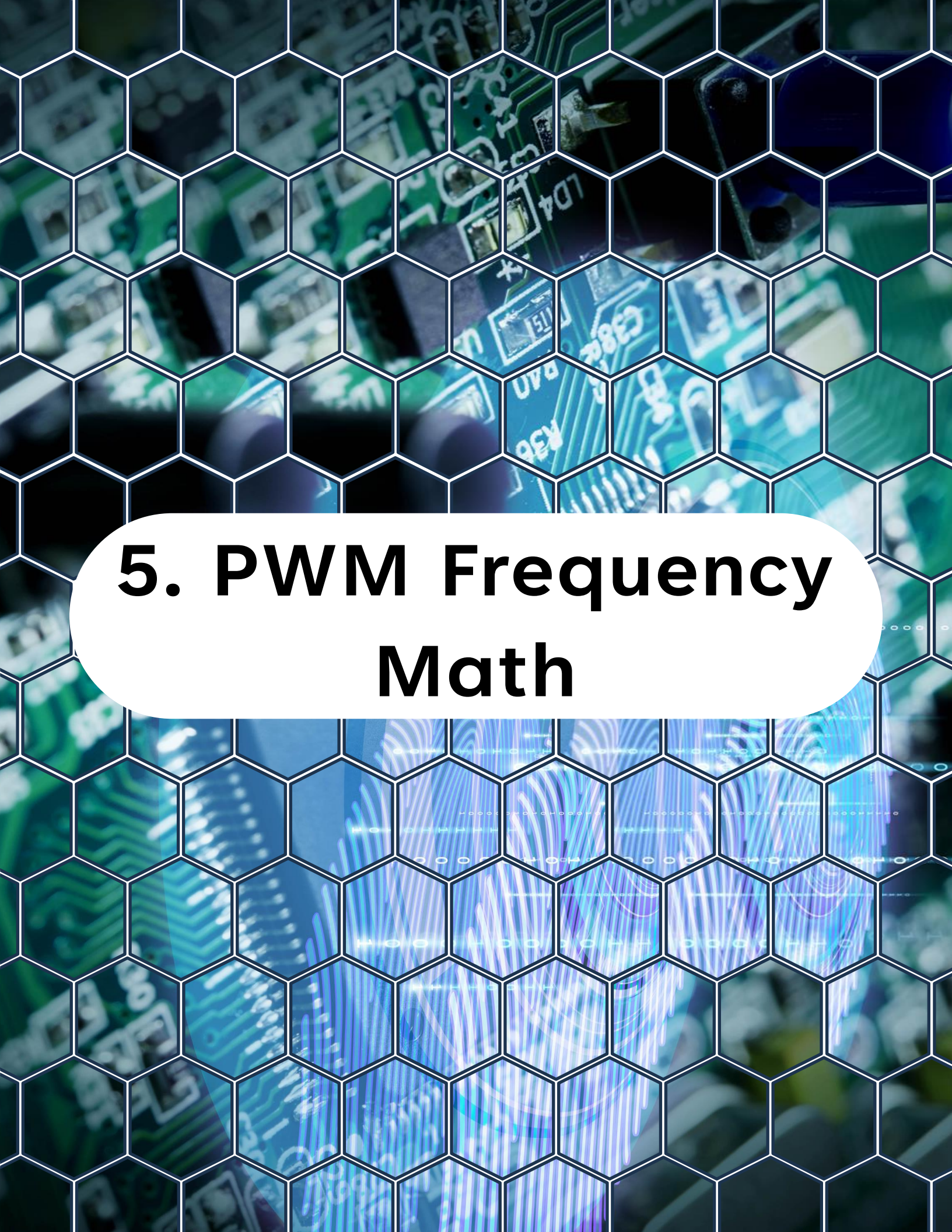
For a basic PWM setup, you will use:

- `TCA0.SINGLE.CTRLA` (enable + clock select)
- `TCA0.SINGLE.CTRLB` (mode select + enable compare outputs)
- `TCA0.SINGLE.PER` (sets PWM period)
- `TCA0.SINGLE.CMP0` (duty for channel 0)
- `TCA0.SINGLE.INTFLAGS` (clear flags if needed)
- `PORTMUX.TCAROUTEA` (route WO pins to actual GPIO pins)

4. TCA Registers You Will Use

You do not need to memorize them. Learn the pattern:

- Choose mode
- Choose period
- Choose compare values
- Route outputs
- Enable timer



5. PWM Frequency Math

5. PWM Frequency Math

TCA counts clock ticks up to **PER**, then wraps.

For single-slope PWM, a practical formula is:

$$\text{PWM_freq} = \text{F_timer} / (\text{PER} + 1)$$

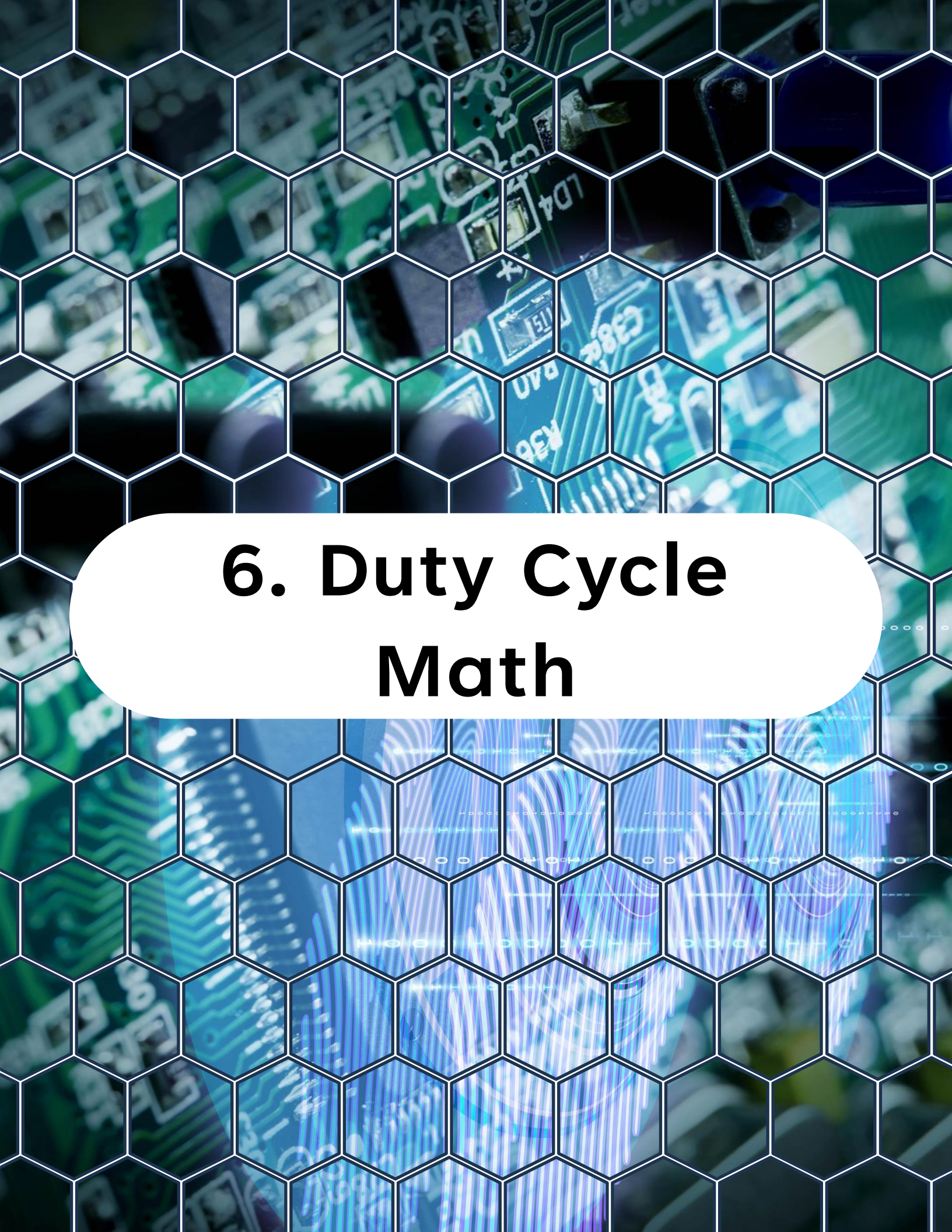
Where:

- **F_timer** is the TCA clock after prescaler
- **PER** is the top value

Example:

- **F_timer** = 1 MHz
- **PER** = 999
- **PWM** = 1 MHz / 1000 = 1 kHz

You will use this formula constantly.



6. Duty Cycle Math

6. Duty Cycle Math

In simplest terms:

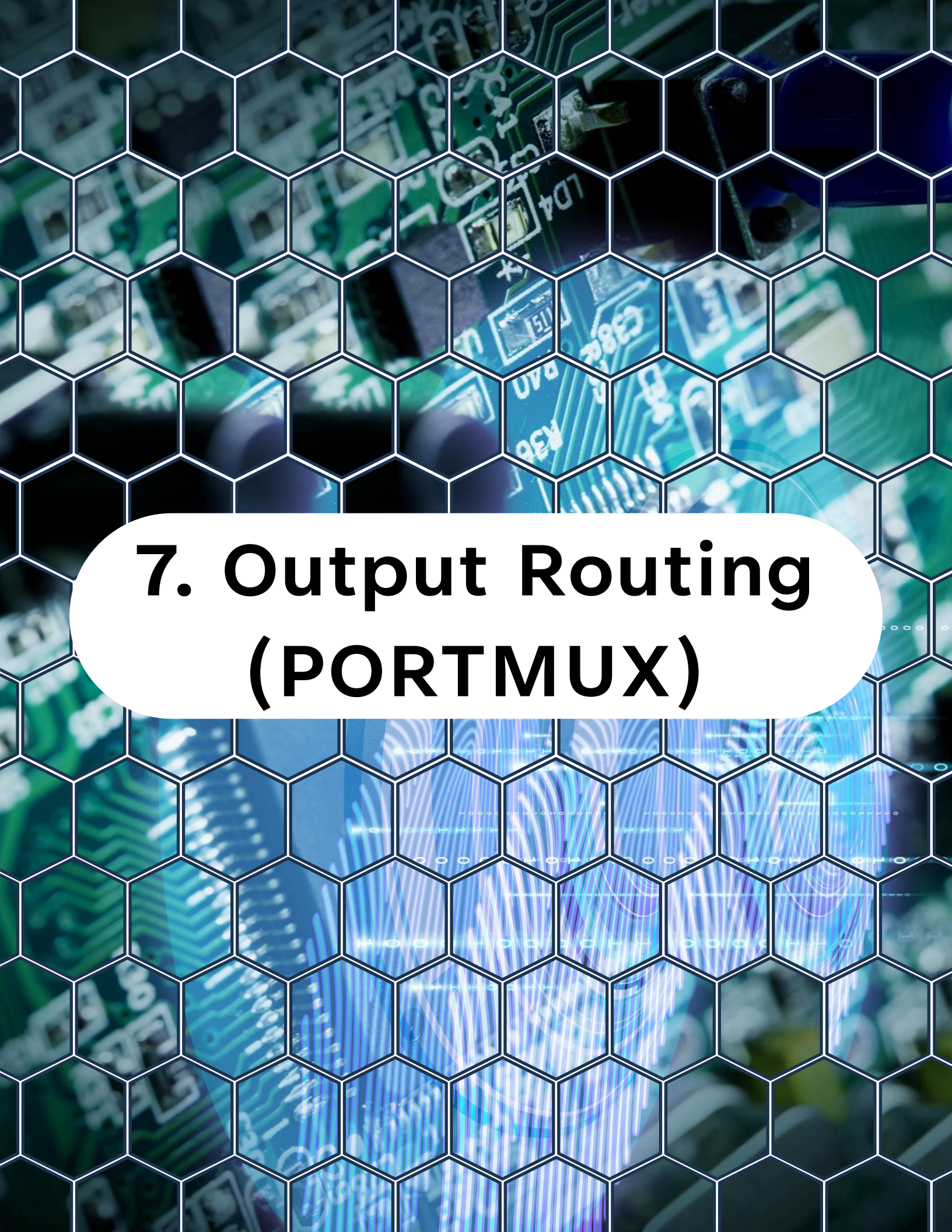
$$\text{Duty} = \text{CMPx} / (\text{PER} + 1)$$

Example:

- **PER** = 999 (1000 counts)
- **CMP0** = 250
- **Duty** = 25%

Rule of thumb:

- **CMP** = 0 -> always low
- **CMP** = **PER** -> almost always high (depends on mode, but effectively high)



7. Output Routing (PORTMUX)

7. Output Routing (PORTMUX)

On DA-series MCUs, many peripherals can route outputs to different pin groups.

TCA waveform outputs are not "magically" connected. You must route them using **PORTMUX**.

This is a common beginner failure:

- Timer is running
- PWM is configured
- Nothing appears on the pin because the output was never routed.

So yes, **PORTMUX** matters.

7. Output Routing (PORTMUX)

TCAn Pin Position

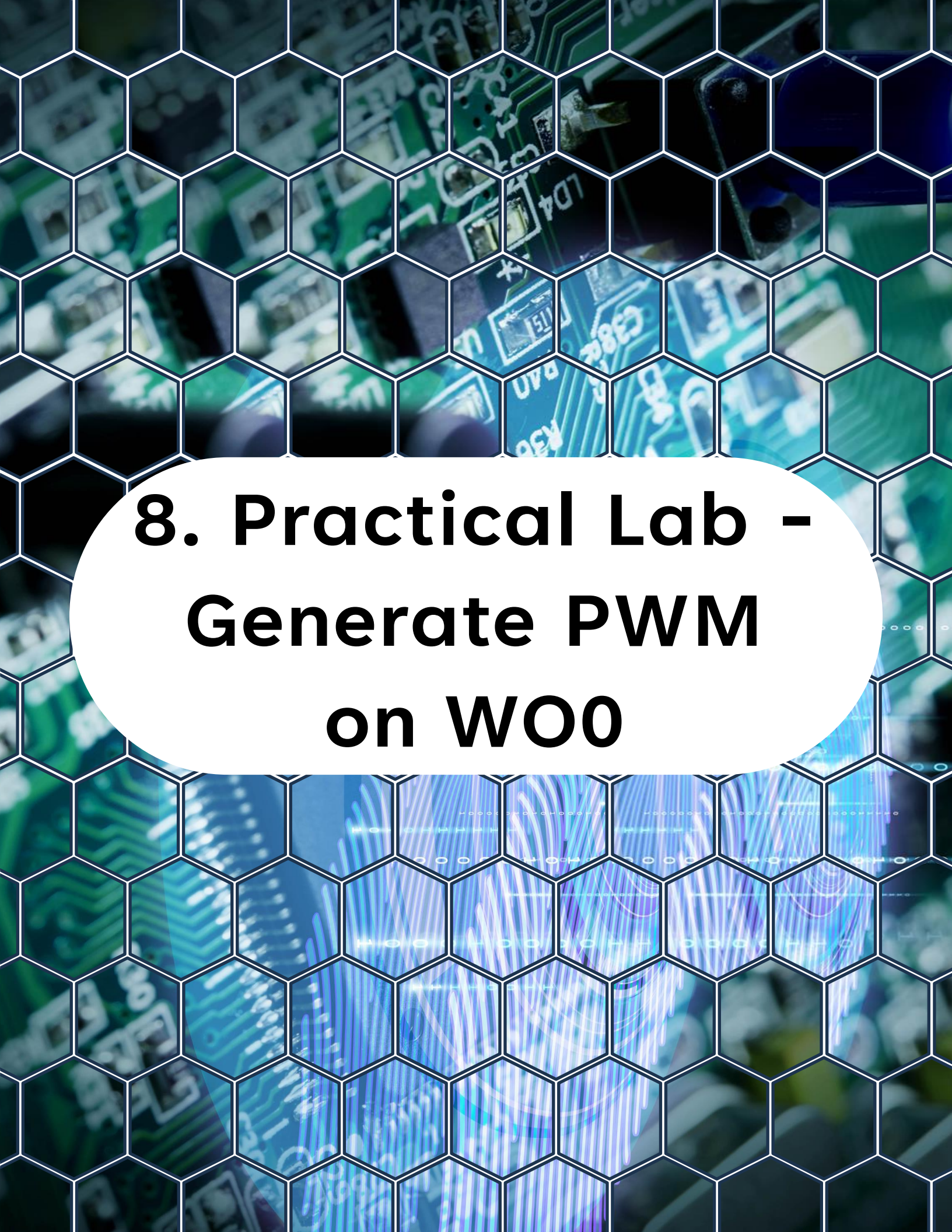
Name: TCAROUTEA
Offset: 0x06
Reset: 0x00
Property: -

Bit	7	6	5	4	3	2	1	0
			TCA1[2:0]			TCA0[2:0]		
Access			R/W	R/W	R/W	R/W	R/W	R/W
Reset			0	0	0	0	0	0

Bits 2:0 – TCA0[2:0] TCA0 Signals

This bit field controls the pin positions for TCA0 signals.

Value	Name	Description					
		WO0	WO1	WO2	WO3	WO4	WO5
0x0	PORTA	PA0	PA1	PA2	PA3	PA4	PA5
0x1	PORTB	PB0	PB1	PB2	PB3	PB4	PB5
0x2	PORTC	PC0	PC1	PC2	PC3	PC4	PC5
0x3	PORTD	PD0	PD1	PD2	PD3	PD4	PD5
0x4	PORTE	PE0	PE1	PE2	PE3	PE4	PE5
0x5	PORTF	PF0	PF1	PF2	PF3	PF4	PF5
0x6	PORTG	PG0	PG1	PG2	PG3	PG4	PG5
0x7	-	Reserved					



8. Practical Lab - Generate PWM on W00

8. Practical Lab - Generate PWM on WO0

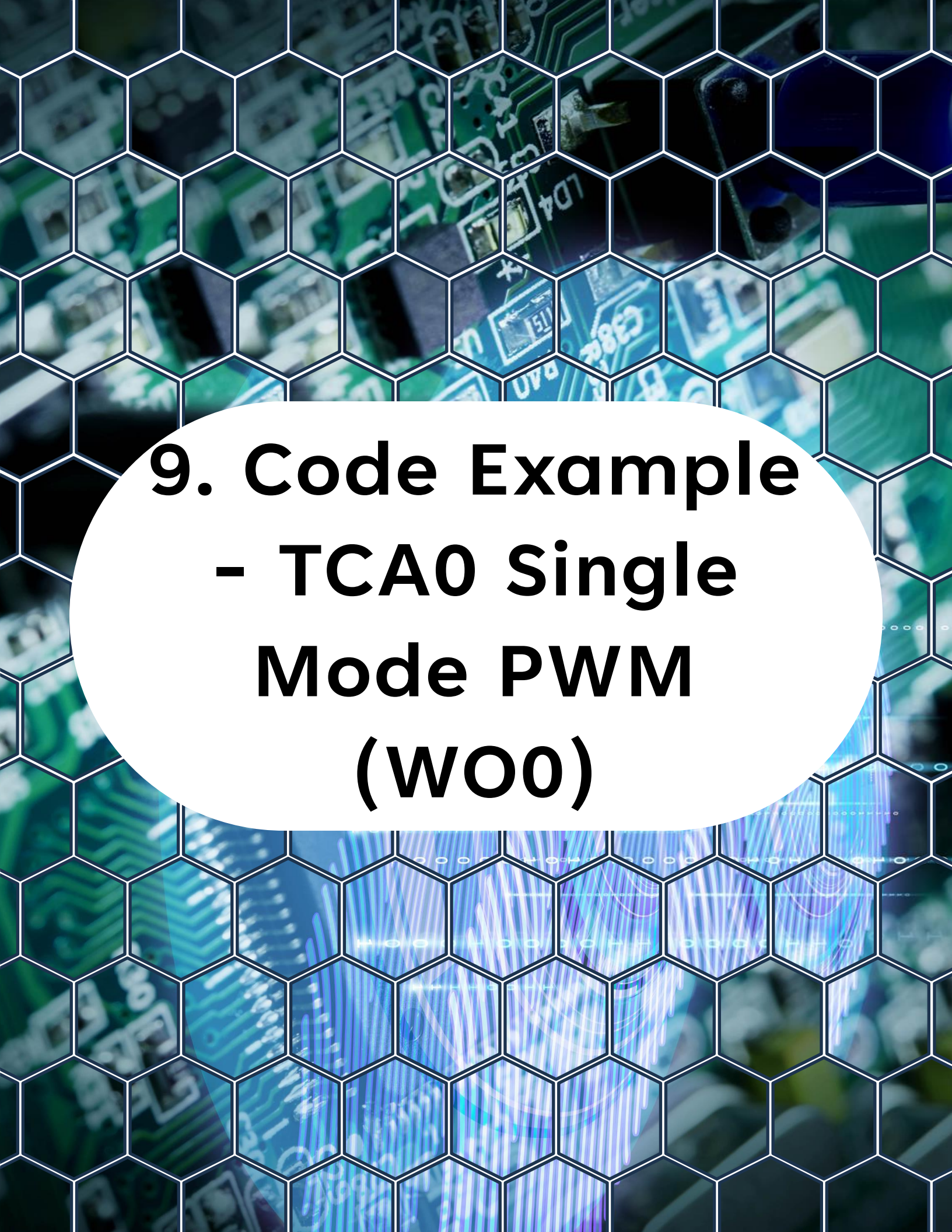
Lab Goal

Generate a PWM waveform on a TCA output pin (WO0), then vary duty cycle.

You can attach:

- Oscilloscope/logic analyzer to the WO pin
or
- Use the onboard LED if it is connected to a compatible pin (varies by board)

Even if LED pin does not match, use a header pin. Measurements are better anyway.



9. Code Example - TCA0 Single Mode PWM (W00)

9. Code Example - TCA0 Single Mode PWM (WO0)

This example:

- Sets up TCA0 Single mode
- Generates PWM on WO0
- Uses a 1 kHz PWM target (example)
- Uses a 50% duty cycle (example)

Important: The exact WO0 pin depends on routing. Adjust pin selection based on your board pinout.

```
1 #include <avr/io.h>
2 #include <stdint.h>
3
4 #define F_CLK_PER_HZ    (24000000UL)
5 #define PWM_TARGET_HZ   (1000UL)
6 #define PWM_DUTY_PC     (50U)
7
8 // Pick one prescaler and stick to it
9 #define TCA_DIV          (64UL)
10 #define TCA_CLKSEL_GC    (TCA_SINGLE_CLKSEL_DIV64_gc)
11
12 // PER = (F_CLK_PER / DIV / F_PWM) - 1
13 #define TCA_PER_VALUE    ((uint16_t)((F_CLK_PER_HZ / TCA_DIV / PWM_TARGET_HZ) - 1UL))
14 #define TCA_CMP_VALUE    ((uint16_t)(((uint32_t)(TCA_PER_VALUE + 1U) * PWM_DUTY_PC) / 100U))
15
16 /**
17  * Initializes TCA0 in SINGLE mode, Single-Slope PWM, enabling WO0 output
```

9. Code Example - TCA0 Single Mode PWM (W00)

```
1  #include <avr/io.h>
2  #include <stdint.h>
3
4  #define F_CLK_PER_HZ      (24000000UL)
5  #define PWM_TARGET_HZ    (10000UL)
6  #define PWM_DUTY_PC      (50U)
7
8  // Pick one prescaler and stick to it
9  #define TCA_DIV           (64UL)
10 #define TCA_CLKSEL_GC     (TCA_SINGLE_CLKSEL_DIV64_gc)
11
12 // PER = (F_CLK_PER / DIV / F_PWM) - 1
13 #define TCA_PER_VALUE     ((uint16_t)((F_CLK_PER_HZ / TCA_DIV / PWM_TARGET_HZ) - 1UL))
14 #define TCA_CMP_VALUE     ((uint16_t)(((uint32_t)(TCA_PER_VALUE + 1U) * PWM_DUTY_PC) / 100U))
15
16 /**
17  * Initializes TCA0 in SINGLE mode, Single-Slope PWM, enabling W00 output,
18  * and the frequency and duty cycle defined.
19  */
20 static void tca0_pwm_w00_init(void)
21 {
22     // Route TCA0 outputs to PORTA (W00 commonly on PA0 on this route)
23     PORTMUX.TCAROUTEA = PORTMUX_TCA0_PORTA_gc;
24
25     // W00 pin (PA0) as output
26     PORTA.DIRSET = PIN0_bm;
27
28     // Stop timer
29     TCA0.SINGLE.CTRLA = 0;
30
31     // Single-slope PWM, enable W00
32     TCA0.SINGLE.CTRLB = (uint8_t)(TCA_SINGLE_WGMODE_SINGLESLOPE_gc |
33     TCA_SINGLE_CMP0EN_bm);
34
35     // Set period and duty
36     TCA0.SINGLE.PER = TCA_PER_VALUE;
37     TCA0.SINGLE.CMP0 = TCA_CMP_VALUE;
38
39     // Start
40     TCA0.SINGLE.CTRLA = (uint8_t)(TCA_CLKSEL_GC | TCA_SINGLE_ENABLE_bm);
41 }
```


9. Code Example - TCA0 Single Mode PWM (WO0)

What to expect:

- A stable PWM signal on WO0 pin
- Frequency close to 10 kHz (depending on F_CPU)
- Duty at ~50%



10. Updating Duty Cycle at Runtime

10. Updating Duty Cycle at Runtime

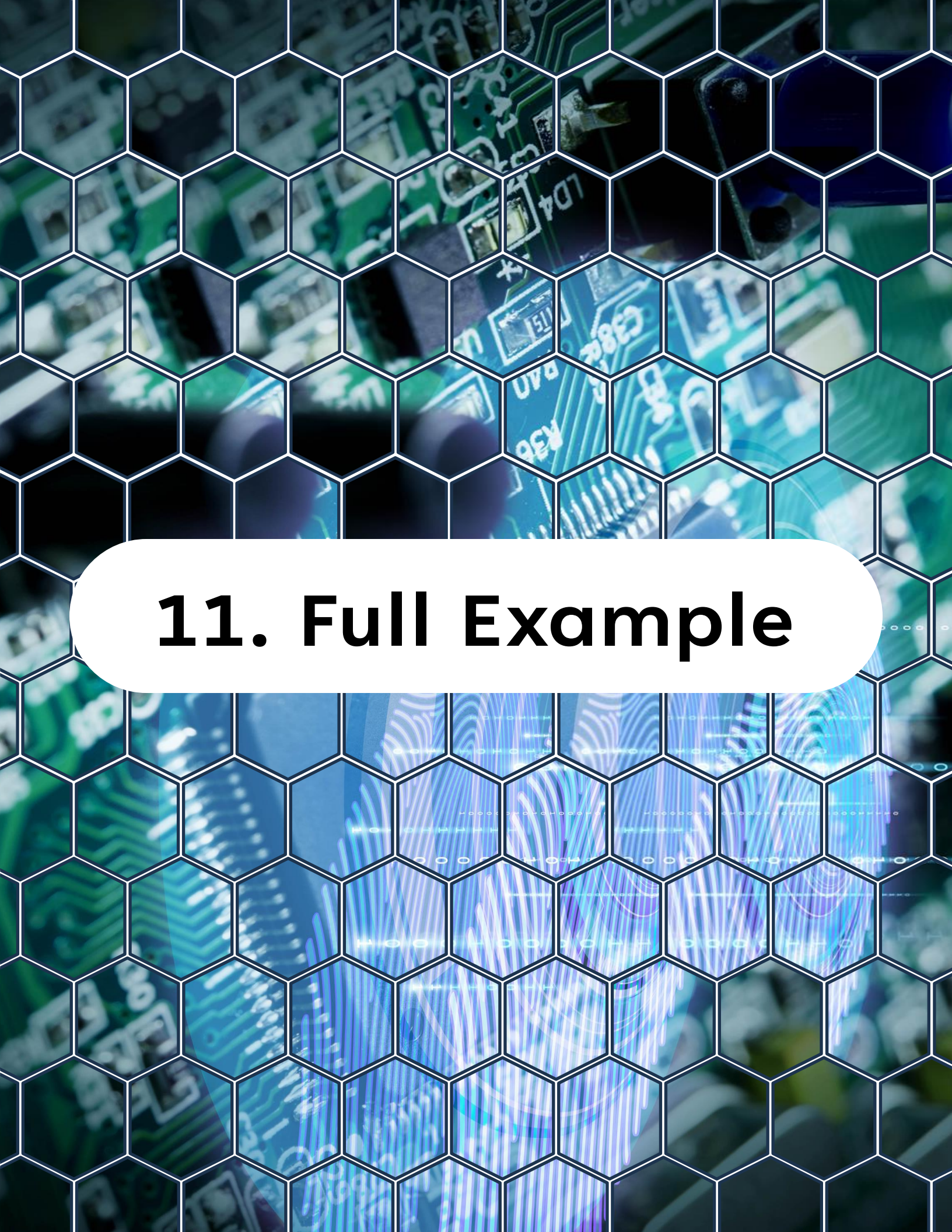
The simplest runtime update is to rewrite CMP0.

This helper sets duty in percent (0 to 100).

```
1 static void tca0_pwm_set_duty_percent(uint8_t duty_percent)
2 {
3     uint32_t top = (uint32_t)TCA0.SINGLE.PER + 1UL;
4     uint32_t cmp = (top * (uint32_t)duty_percent) / 100UL;
5
6     if (cmp > 0)
7     {
8         cmp -= 1; /* keep within range for edge cases */
9     }
10
11     TCA0.SINGLE.CMP0 = (uint16_t)cmp;
12 }
```

Notes:

- This is fine for learning and many real projects.
- Later, you will learn about buffered updates and synchronization (especially important at high frequencies).



11. Full Example

11. Full Example

```
1 #include <avr/io.h>
2 #include <stdint.h>
3 #include <util/delay.h>
4
5 #define F_CLK_PER_HZ    (24000000UL)
6 #define PWM_TARGET_HZ   (10000UL)
7 #define PWM_DUTY_PC     (50U)
8
9 // Pick one prescaler and stick to it
10 #define TCA_DIV          (64UL)
11 #define TCA_CLKSEL_GC    (TCA_SINGLE_CLKSEL_DIV64_gc)
12
13 // PER = (F_CLK_PER / DIV / F_PWM) - 1
14 #define TCA_PER_VALUE    ((uint16_t)((F_CLK_PER_HZ / TCA_DIV / PWM_TARGET_HZ) - 1UL))
15 #define TCA_CMP_VALUE    ((uint16_t)(((uint32_t)(TCA_PER_VALUE + 1U) * PWM_DUTY_PC) / 100U))
16
17 /**
18  * Initializes TCA0 in SINGLE mode, Single-Slope PWM, enabling W00 output,
19  * and the frequency and duty cycle defined.
20  */
21 static void tca0_pwm_wo0_init(void)
22 {
23     // Route TCA0 outputs to PORTA (W00 commonly on PA0 on this route)
24     PORTMUX.TCAROUTEA = PORTMUX_TCA0_PORTA_gc;
25
26     // W00 pin (PA0) as output
27     PORTA.DIRSET = PIN0_bm;
28
29     // Stop timer
30     TCA0.SINGLE.CTRLA = 0;
31
32     // Single-slope PWM, enable W00
33     TCA0.SINGLE.CTRLB = (uint8_t)(TCA_SINGLE_WGMODE_SINGLESLOPE_gc |
34     TCA_SINGLE_CMP0EN_bm);
35
36     // Set period and duty
37     TCA0.SINGLE.PER  = TCA_PER_VALUE;
38     TCA0.SINGLE.CMP0 = TCA_CMP_VALUE;
39
40     // Start
41     TCA0.SINGLE.CTRLA = (uint8_t)(TCA_CLKSEL_GC | TCA_SINGLE_ENABLE_bm);
42 }
43
44 /**
45  * Helper to sets duty in percent (0 to 100).
46  */
```

11. Full Example

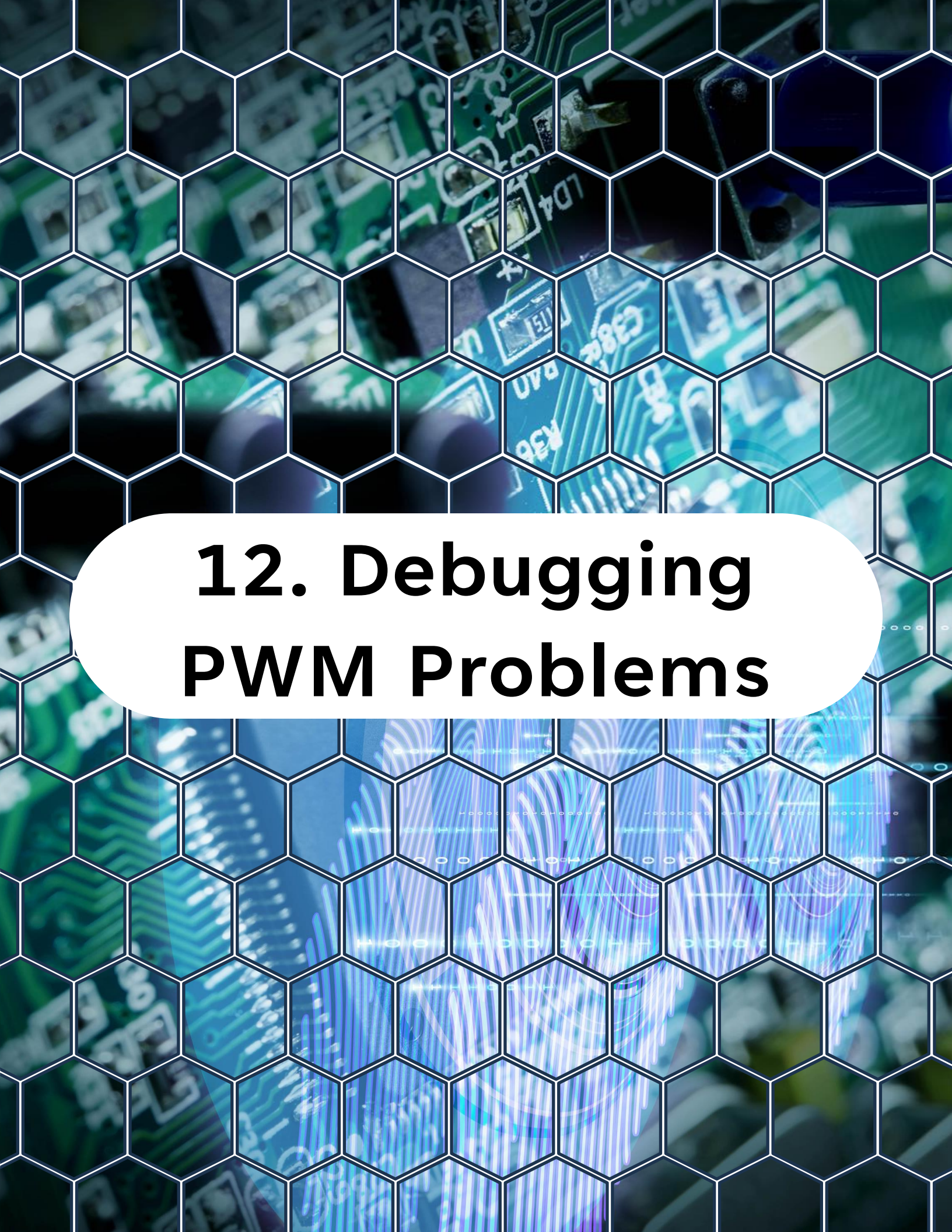
```
44 /**
45  * Helper to sets duty in percent (0 to 100).
46  */
47 static void tca0_pwm_set_duty_percent(uint8_t duty_percent)
48 {
49     uint32_t top = (uint32_t)TCA0.SINGLE.PER + 1UL;
50     uint32_t cmp = (top * (uint32_t)duty_percent) / 100UL;
51
52     if (cmp > 0)
53     {
54         cmp -= 1; /* keep within range for edge cases */
55     }
56
57     TCA0.SINGLE.CMP0 = (uint16_t)cmp;
58 }
59
60 /**
61  * Writes to a CCP-protected I/O register.
62  *
63  * Args:
64  *   addr: Pointer to CCP-protected register.
65  *   value: Value to write.
66  */
67 static inline void ccp_write_io(volatile uint8_t *addr, uint8_t value)
68 {
69     CCP = CCP_IOREG_gc;
70     *addr = value;
71 }
72
73 /**
74  * Configures the main clock source to OSCHF and sets OSCHF to 24 MHz.
75  * Disables the main prescaler.
76  */
77 static void clock_to_24mhz_no_prescale(void)
78 {
79     // OSCHF = 24 MHz
80     ccp_write_io(&CLKCTRL.OSCHFCTRLA, CLKCTRL_FRQSEL_24M_gc);
81
82     // Main clock source = OSCHF
83     ccp_write_io(&CLKCTRL.MCLKCTRLA, CLKCTRL_CLKSEL_OSCHF_gc);
84
85     // Disable prescaler so CLK_PER = 24 MHz
86     ccp_write_io(&CLKCTRL.MCLKCTRLB, 0x00);
87 }
88
89 /**
90  * Application entry point.
91  */
```


11. Full Example

```
88
89 /**
90  * Application entry point.
91  */
92 int main(void)
93 {
94     // Configures the main clock
95     clock_to_24mhz_no_prescale();
96
97     // Initializes TCA0
98     tca0_pwm_wo0_init();
99
100    while (1) {
101        // Ramp duty cycle up and down for visibility
102        for (uint8_t d = 0; d <= 100; d += 5)
103        {
104            tca0_pwm_set_duty_percent(d);
105            _delay_ms(50);
106        }
107
108        for (int8_t d = 100; d >= 0; d -= 5)
109        {
110            tca0_pwm_set_duty_percent((uint8_t)d);
111            _delay_ms(50);
112        }
113    }
114 }
```

Yes, this uses delay. That is OK for a visible demo.

In later days, you will replace this with timer-driven ramping.



12. Debugging PWM Problems

12. Debugging PWM Problems

If you see no waveform:

- Verify the TCA clock is running (Day 3)
- Verify TCA0 is enabled (CTRLA ENABLE bit)
- Verify WGMODE is set to single-slope PWM
- Verify CMP0EN is enabled
- Verify the WO0 pin is configured as output
- Verify PORTMUX routing matches the pin you are probing

If frequency is wrong:

- Confirm **F_CPU**
- Confirm **prescaler**
- Recalculate **PER**

If duty is wrong:

- Confirm **CMP0** range relative to **PER**



13. What You Learned Today

13. What You Learned Today

By the end of Day 7, you understand:

- How PWM works physically
- How TCA Single mode generates PWM
- How PER sets frequency
- How CMPx sets duty cycle
- How to route waveform outputs to pins (PORTMUX)
- How to update duty cycle safely in firmware

This is one of the most reusable skills in embedded systems.



14. What Comes Next

14. What Comes Next

Day 8 - TCA Split Mode

Next, you will:

- Split TCA into two 8-bit timers
- Generate more independent PWM channels
- Understand tradeoffs: resolution vs flexibility
- Build patterns for multi-channel PWM control

When you are ready, we continue with Day 8.

"Thanks for watching!

If you enjoyed this video, make sure to hit the like button and subscribe to stay updated with my latest content.

Don't forget to check out my other videos for more tips and tutorials on Embedded C, Python, hardware designs, etc. Keep exploring, keep learning, and I'll see you in the next video!"

<https://www.linkedin.com/in/yamil-garcia>

<https://www.youtube.com/@LearningByTutorials>

<https://github.com/god233012yamil>